

Các thao tác cơ bản và ứng dụng của cây splay

Yang Siyu

Nguồn gốc: Basic operations and applications of splay trees

IOI China candidate team papers / 2004 / Yang Siyu.pdf

Từ khóa

Cây splay; thao tác cơ bản; ứng dụng.

Tóm tắt

Bài viết này giới thiệu các thao tác cơ bản của cây splay và cách dùng nó trong giải bài tập. Nội dung chính có thể chia thành bốn phần.

Phần mở đầu nêu vai trò quan trọng của cây tìm kiếm nhị phân trong thi lập trình, đồng thời chỉ ra rằng trong một số trường hợp độ phức tạp thời gian của cây tìm kiếm nhị phân có thể rất cao. Từ đó ta dẫn tới cây splay, một biến thể có độ phức tạp trung bình theo nghĩa khấu hao tốt hơn.

Phần thứ hai trình bày các thao tác cơ bản của cây splay, kèm phân tích và chứng minh độ phức tạp thời gian. Kết quả là mọi thao tác cơ bản của cây splay đều có độ phức tạp khấu hao $O(\log n)$; vì vậy cây splay có thể xem là một loại cây tìm kiếm nhị phân cân bằng theo nghĩa khấu hao.

Phần thứ ba dùng một ví dụ để minh họa ứng dụng của cây splay trong giải bài, và so sánh nó với một số cấu trúc dữ liệu dạng cây khác.

Phần cuối cùng tổng kết các ưu điểm của cây splay.

1 Mở đầu

Cây tìm kiếm nhị phân có thể hỗ trợ nhiều thao tác trên tập động. Vì vậy, trong thi tin học, cây tìm kiếm nhị phân giữ một vị trí rất quan trọng: nó có thể dùng để biểu diễn tập có thứ tự, xây dựng chỉ mục, cài đặt hàng đợi ưu tiên, v.v.

Thời gian của các thao tác cơ bản trên cây tìm kiếm nhị phân tỉ lệ với chiều cao của cây. Với một cây nhị phân hoàn chỉnh có n nút, thời gian xấu nhất của những thao tác này là $O(\log n)$. Nhưng nếu cây có n nút lại suy biến thành một dãy tuyến, thời gian xấu nhất sẽ là $O(n)$. Một số biến thể của cây tìm kiếm nhị phân, như cây đỏ đen hay cây AVL, vẫn bảo đảm hiệu năng tốt trong trường hợp xấu nhất.

Cây splay cũng là một cải tiến của cây tìm kiếm nhị phân. Nó không bảo đảm rằng cây luôn “cân bằng”, nhưng với một dãy thao tác trên cây splay, ta có thể chứng minh chi phí khấu hao của mỗi thao tác đều là $O(\log n)$. Theo nghĩa đó, cây splay cũng là một cây tìm kiếm nhị phân cân bằng. Trong các cấu trúc dữ liệu dạng cây, cây splay còn có ưu điểm tốt về yêu cầu bộ nhớ và độ phức tạp khi lập trình.

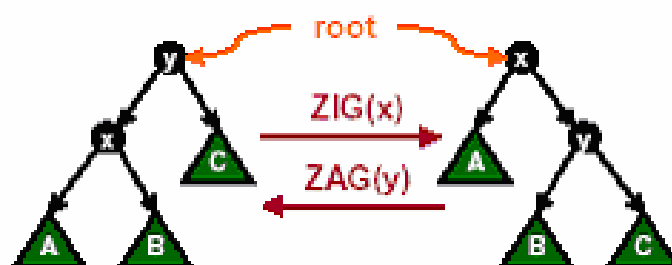
2 Các thao tác cơ bản của cây splay

Cây splay là một cải tiến của cây tìm kiếm nhị phân, nên nó cũng giữ tính thứ tự: với mỗi nút x , mọi phần tử trong cây con trái của x đều nhỏ hơn x , và mọi phần tử trong cây con phải của x đều lớn hơn x . Điểm khác với cây tìm kiếm nhị phân thông thường là cây splay có khả năng tự điều chỉnh, nhờ thao tác splay $\text{Splay}(x, S)$.

2.1 Thao tác splay

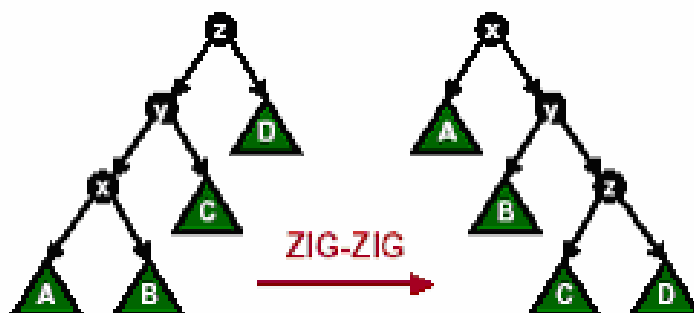
Thao tác $\text{Splay}(x, S)$ đưa phần tử x trong cây splay S lên gốc bằng một dãy phép quay, trong khi vẫn giữ nguyên tính thứ tự của cây. Trong quá trình điều chỉnh, ta xét ba trường hợp.

Trường hợp 1. Cha y của x là gốc. Nếu x là con trái của y , ta thực hiện một phép **Zig**, tức quay phải tại y ; nếu x là con phải của y , ta thực hiện một phép **Zag**, tức quay trái tại y . Sau phép quay, x trở thành gốc của cây S và thao tác kết thúc.



Hình 1: Trường hợp Zig/Zag: cha của x là gốc, nên một phép quay đơn đưa x lên gốc.

Trường hợp 2. Cha y của x không phải là gốc; gọi z là cha của y . Nếu x và y cùng là con trái của cha mình, hoặc cùng là con phải của cha mình, ta thực hiện một bước **Zig-Zig** hoặc **Zag-Zag**. Nghĩa là ta quay ở nút z , rồi tiếp tục quay ở nút y theo cùng một hướng.



Hình 2: Trường hợp Zig-Zig/Zag-Zag: x và y nằm cùng phía so với cha của mình, nên thực hiện hai phép quay cùng hướng.

Trường hợp 3. Cha y của x không phải là gốc; gọi z là cha của y . Nếu một trong hai nút x, y là con trái của cha mình còn nút kia là con phải, ta thực hiện **Zig-Zag** hoặc **Zag-Zig**. Nghĩa là ta quay ở y trước, rồi quay ở z theo hướng ngược lại.

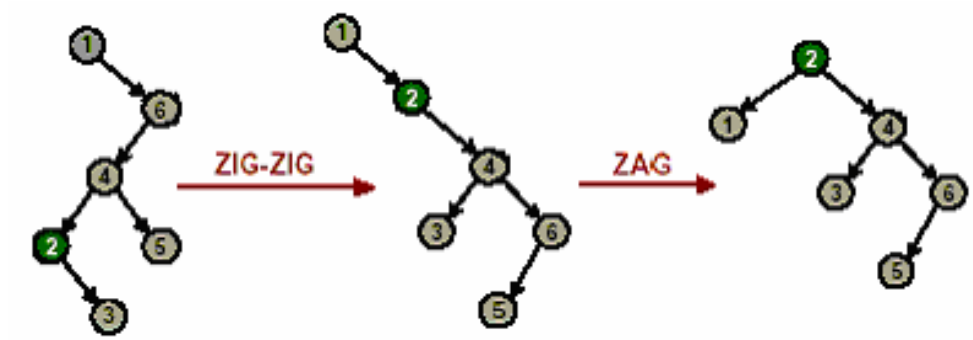


Hình 3: Trường hợp Zig-Zag/Zag-Zig: x và y nằm khác phía, nên hai phép quay có hướng ngược nhau.

Ví dụ, khi thực hiện $\text{Splay}(1, S)$, phần tử 1 được đưa lên gốc của cây splay S . Nếu sau đó thực hiện $\text{Splay}(2, S)$, cây thường trở nên cân bằng hơn theo trực giác. Bản thân quá trình splay không phức tạp: ở mỗi bước chỉ cần phân loại vị trí của nút hiện tại rồi thực hiện các phép quay tương ứng. Ba kiểu quay trên đều được tạo từ hai phép quay cơ bản là quay trái và quay phải, nên việc cài đặt khá đơn giản.



Hình 4: Ví dụ thực hiện $\text{Splay}(1, S)$: các bước Zig-Zig rồi Zig đưa nút 1 lên gốc.



Hình 5: Tiếp tục thực hiện $\text{Splay}(2, S)$ sau ví dụ trước: cây trở nên cân bằng hơn theo trực giác.

2.2 Các thao tác cơ bản

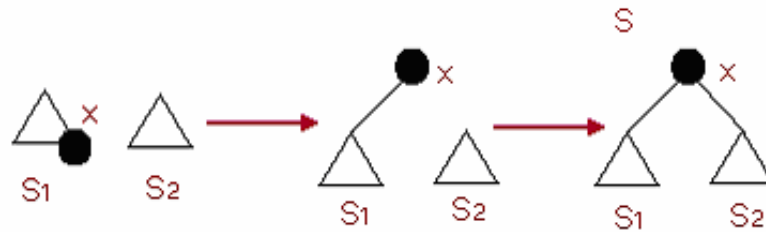
Dựa vào thao tác splay, ta có thể thực hiện các phép toán sau trên cây splay S .

Find(x, S). Kiểm tra xem phần tử x có thuộc tập có thứ tự được biểu diễn bởi S hay không. Trước hết ta tìm x giống như trên cây tìm kiếm nhị phân. Nếu x có trong cây, ta tiếp tục thực hiện $\text{Splay}(x, S)$ để điều chỉnh cây.

Insert(x, S). Chèn phần tử x vào tập có thứ tự được biểu diễn bởi S . Trước hết ta chèn x vào đúng vị trí như trong cây tìm kiếm nhị phân thông thường, sau đó thực hiện $\text{Splay}(x, S)$.

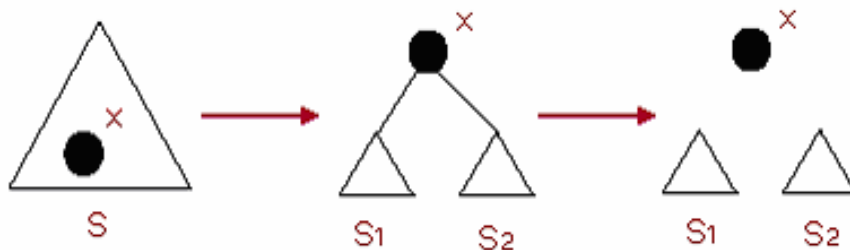
Delete(x, S). Xóa phần tử x khỏi tập có thứ tự được biểu diễn bởi S . Trước hết ta tìm vị trí của x bằng cách tìm kiếm trong cây nhị phân. Nếu x không có con hoặc chỉ có một con, ta xóa trực tiếp x , rồi dùng thao tác splay đưa cha của x lên gốc. Nếu x có hai con, ta tìm nút kế tiếp y của x , dùng y thay cho vị trí của x , rồi thực hiện $\text{Splay}(y, S)$ để đưa y lên gốc.

Join(S_1, S_2). Hợp nhất hai cây splay S_1 và S_2 thành một cây splay, với điều kiện mọi phần tử của S_1 đều nhỏ hơn mọi phần tử của S_2 . Ta tìm phần tử lớn nhất x trong S_1 , dùng $\text{Splay}(x, S_1)$ để đưa x lên gốc của S_1 , rồi gắn S_2 làm cây con phải của x . Kết quả thu được là cây splay mới S .



Hình 6: Thao tác Join: đưa phần tử lớn nhất của S_1 lên gốc rồi gắn S_2 làm cây con phải.

Split(x, S). Dùng x làm mốc để tách S thành hai cây splay S_1 và S_2 , trong đó mọi phần tử của S_1 nhỏ hơn x , còn mọi phần tử của S_2 lớn hơn x . Trước hết ta thực hiện $\text{Find}(x, S)$ để đưa x lên gốc của cây. Khi đó cây con trái của x chính là S_1 , còn cây con phải là S_2 .



Hình 7: Thao tác Split: đưa x lên gốc rồi tách hai cây con trái và phải thành S_1 và S_2 .

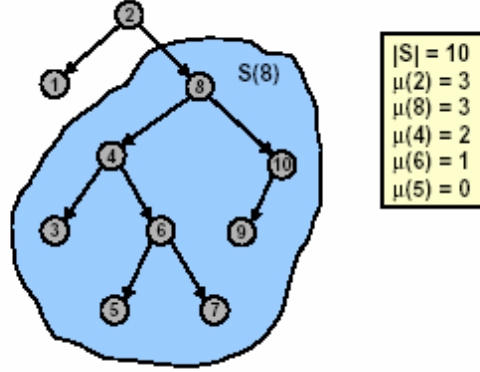
Ngoài năm thao tác trên, cây splay còn hỗ trợ tìm giá trị lớn nhất, nhỏ nhất, tiền nhiệm, kế nhiệm, v.v. Những thao tác này cũng đều xây dựng trên nền tảng của thao tác splay.

2.3 Phân tích độ phức tạp thời gian

Từ cách cài đặt các thao tác trên, có thể thấy hiệu quả thời gian của chúng phụ thuộc hoàn toàn vào độ phức tạp của thao tác splay. Ta sẽ dùng phương pháp kế toán để phân tích chi phí khấu hao của splay.

Trước hết định nghĩa một số ký hiệu. $S(x)$ là cây con gốc x ; $|S|$ là số nút của cây splay S . Đặt

$$\mu(S) = \lfloor \log |S| \rfloor, \quad \mu(x) = \mu(S(x)).$$



Hình 8: Ví dụ ký hiệu $S(x)$ và các giá trị $\mu(x)$ trong phân tích khấu hao của thao tác splay.

Ta dùng 1 đồng để biểu diễn một đơn vị chi phí, trong đó một lần truy cập vào một nút hoặc một phép quay được xem là chi phí đơn vị. Bất biến của cây splay được định nghĩa như sau: tại mọi thời điểm, mỗi nút x trong cây phải có ít nhất $\mu(x)$ đồng tiền dự trữ.

Trong quá trình splay, chi phí được dùng cho hai việc:

1. Trả cho thời gian sử dụng, tức mỗi thao tác đơn vị cần trả 1 đồng.
2. Khi hình dạng cây thay đổi, ta phải nạp thêm tiền hoặc phân phối lại số tiền đang có ở các nút để bất biến tiếp tục đúng.

Ta có định lý sau về chi phí của thao tác splay.

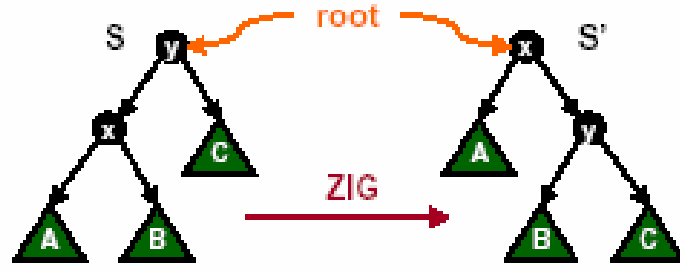
Định lý. Trong mỗi thao tác $\text{Splay}(x, S)$, tổng chi phí để điều chỉnh cấu trúc cây và duy trì bất biến không vượt quá $3\mu(S) + 1$.

Chứng minh. Ký hiệu $\mu(x)$ và $\mu'(x)$ lần lượt là lượng tiền tại nút x trước và sau một bước Zig, Zig-Zig hoặc Zig-Zag. Ta xét ba trường hợp.

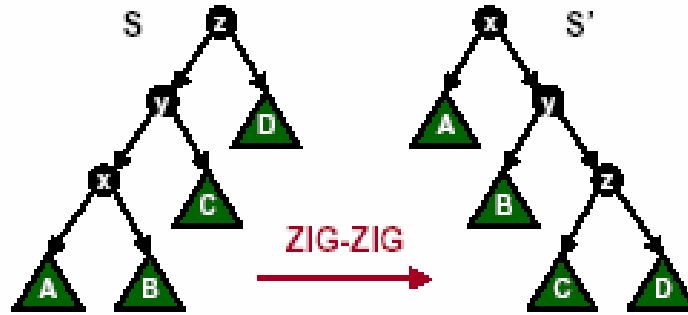
Trường hợp 1: Zig hoặc Zag. Để duy trì bất biến của cây splay sau phép quay, chi phí thay đổi tiền dự trữ là

$$\begin{aligned} \mu'(x) + \mu'(y) - \mu(x) - \mu(y) &= \mu'(y) - \mu(x) \\ &\leq \mu'(x) - \mu(x) \\ &\leq 3(\mu'(x) - \mu(x)) \\ &= 3(\mu(S) - \mu(x)). \end{aligned}$$

Ngoài ra ta cần thêm 1 đồng để trả cho thao tác truy cập và quay. Vì vậy một bước Zig hoặc Zag tốn nhiều nhất $3(\mu(S) - \mu(x)) + 1$.



Hình 9: Ký hiệu trước và sau một bước Zig trong chứng minh chi phí khấu hao.



Hình 10: Ký hiệu trước và sau một bước Zig-Zig trong chứng minh chi phí khấu hao.

Trường hợp 2: Zig-Zig hoặc Zag-Zag. Để duy trì bất biến, chi phí thay đổi tiền dự trữ là

$$\begin{aligned}
 & \mu'(x) + \mu'(y) + \mu'(z) - \mu(x) - \mu(y) - \mu(z) \\
 &= \mu'(y) + \mu'(z) - \mu(x) - \mu(y) \\
 &= (\mu'(y) - \mu(x)) + (\mu'(z) - \mu(y)) \\
 &\leq (\mu'(x) - \mu(x)) + (\mu'(x) - \mu(x)) \\
 &= 2(\mu'(x) - \mu(x)).
 \end{aligned}$$

Giống trường hợp trước, ta còn phải trả 1 đồng cho thao tác đơn vị. Nếu $\mu'(x) > \mu(x)$, thì

$$2(\mu'(x) - \mu(x)) + 1 \leq 3(\mu'(x) - \mu(x)).$$

Vì vậy chi phí của bước Zig-Zig không vượt quá $3(\mu'(x) - \mu(x))$.

Nếu $\mu'(x) = \mu(x)$, ta chứng minh được

$$\mu'(x) + \mu'(y) + \mu'(z) < \mu(x) + \mu(y) + \mu(z).$$

Nghĩa là không cần nạp thêm tiền để duy trì bất biến, thậm chí còn thu hồi được tiền; dùng 1 đồng trong phần thu hồi này để trả cho thao tác truy cập và quay. Để thấy điều đó, giả sử ngược lại rằng về trái lớn hơn về phải. Khi đó ta có $\mu(x) = \mu'(x) = \mu(z)$, suy ra $\mu(x) = \mu(y) = \mu(z)$ và $\mu(x) = \mu'(z) = \mu(z)$. Đặt $a = 1 + |A| + |B|$ và $b = 1 + |C| + |D|$, ta được

$$\lfloor \log a \rfloor = \lfloor \log b \rfloor = \lfloor \log(a + b + 1) \rfloor. \quad (1)$$

Không mất tính tổng quát, giả sử $b \geq a$. Khi đó

$$\lfloor \log(a + b + 1) \rfloor \geq \lfloor \log(2a) \rfloor = 1 + \lfloor \log a \rfloor > \lfloor \log a \rfloor, \quad (2)$$

mâu thuẫn với (1). Do đó khi $\mu'(x) = \mu(x)$, bước Zig-Zig không cần chi phí thêm, và hiển nhiên cũng không vượt quá $3(\mu'(x) - \mu(x))$.

Trường hợp 3: Zig-Zag hoặc Zag-Zig. Tương tự trường hợp thứ hai, có thể chứng minh mỗi bước Zig-Zag cũng tốn không quá $3(\mu'(x) - \mu(x))$.

Ba trường hợp trên cho thấy một bước Zig có chi phí tối đa $3(\mu(S) - \mu(x)) + 1$, còn một bước Zig-Zig hoặc Zig-Zag có chi phí tối đa $3(\mu'(x) - \mu(x))$. Cộng chi phí của các phép quay theo thứ tự thực hiện, ta thấy một thao tác Splay(x, S) không tốn quá $3\mu(S) + 1$. Do đó mọi thao tác cơ bản dựa trên splay đều có độ phức tạp khấu hao $O(\log n)$. Vì vậy cây splay là một cấu trúc dữ liệu có hiệu quả thời gian rất tốt.

3 Ứng dụng của cây splay

Là một cấu trúc dữ liệu có hiệu quả thời gian cao và yêu cầu bộ nhớ nhỏ, cây splay có nhiều đất dụng võ trong giải bài. Ta xét ví dụ sau.

Ví dụ: Thống kê doanh thu Turnover

Bài toán xuất phát từ kỳ tuyển chọn đội Hồ Nam năm 2002.

Tóm tắt đề bài. Tiger vừa được công ty bổ nhiệm làm trưởng phòng kinh doanh. Nhiệm vụ đầu tiên của anh là thống kê và phân tích tình hình kinh doanh của công ty từ ngày thành lập. Sổ sách ghi lại doanh thu của từng ngày. Do ngày lễ, giảm giá lớn hoặc các nguyên nhân khác, doanh thu có thể dao động; dao động vừa phải là chấp nhận được, nhưng nếu một ngày doanh thu tăng hoặc giảm đột ngột thì có thể tình hình kinh doanh đang có vấn đề. Trong quản lý kinh tế, người ta dùng giá trị dao động nhỏ nhất của một ngày để đo hiện tượng này:

dao động nhỏ nhất của ngày hiện tại = $\min\{|\text{doanh thu một ngày trước đó} - \text{doanh thu ngày hiện tại}|\}$.

Dao động nhỏ nhất càng lớn thì tình hình kinh doanh càng kém ổn định. Để phân tích mức ổn định của toàn bộ công ty từ khi thành lập đến nay, chỉ cần cộng các giá trị dao động nhỏ nhất của từng ngày. Nhiệm vụ là viết chương trình giúp Tiger tính tổng này.

Ngày đầu tiên có giá trị dao động nhỏ nhất bằng chính doanh thu của ngày đó. Dữ liệu có $n \leq 32767$, doanh thu mỗi ngày $a_i \leq 1\,000\,000$, và kết quả cuối cùng $T \leq 2^{31}$.

Phân tích ban đầu. Ý nghĩa của đề rất rõ: mỗi lần đọc một số mới, ta cần tìm trong các số đã đọc trước đó một số có khoảng cách nhỏ nhất với nó.

Ta dễ nghĩ tới thuật toán $O(n^2)$: mỗi lần đọc một số, duyệt lại tất cả các số trước đó để lấy hiệu nhỏ nhất với số hiện tại rồi cộng vào kết quả T . Nhưng vì n lớn, cách này không thể chạy kịp thời hạn. Nếu dùng cây đoạn để ghi nhận các giá trị đã xuất hiện, ta cần một mảng cỡ $2M$, điều này không khả thi trong môi trường Turbo Pascal 7.0 của cuộc thi khi đó. Cây đỏ đen và cây AVL có thời gian và bộ nhớ tốt, nhưng độ phức tạp lập trình cao. Vì vậy cây splay trở thành một lựa chọn tự nhiên.

Mô tả thuật toán. Phân tích kỹ hơn, bài toán chỉ cần ba thao tác trên tập có thứ tự: chèn, tìm tiền nhiệm và tìm kế nhiệm. Cây splay có độ phức tạp rất tốt cho cả ba thao tác này, nên ta thiết kế thuật toán như sau.

Ban đầu cây S rỗng, tổng T bằng 0. Mỗi lần đọc một số p , thực hiện Insert(p, S) để chèn p vào cây splay S . Sau bước này, p được đưa lên gốc. Khi đó nút phải nhất trong cây con trái của p là tiền nhiệm

của p , còn nút trái nhất trong cây con phải của p là kế nhiệm của p trong tập có thứ tự. Lấy hiệu nhỏ nhất giữa p với hai giá trị này rồi cộng vào kết quả T .

Nhận xét. Vì các thao tác cơ bản của cây splay đều có độ phức tạp khấu hao $O(\log n)$, toàn bộ thuật toán có độ phức tạp thời gian $O(n \log n)$ và có thể chạy trong thời hạn. Về bộ nhớ, ta có thể dùng mảng để mô phỏng con trỏ và lưu cấu trúc cây; lượng bộ nhớ không quá 400 KB, đủ để dùng bộ nhớ động trong Pascal. Về lập trình, cây splay khá đơn giản, chương trình không quá phức tạp. Dù không phải thuật toán duy nhất cho bài này, cây splay vẫn có nhiều ưu thế so với các cấu trúc dữ liệu thường dùng khác.

	Tìm tuần tự	Cây đoạn	Cây AVL	Cây splay
Độ phức tạp thời gian	$O(n^2)$	$O(n \log a)$	$O(n \log n)$	$O(n \log n)$
Độ phức tạp bộ nhớ	$O(n)$	$O(a)$	$O(n)$	$O(n)$
Độ phức tạp lập trình	rất đơn giản	khá đơn giản	khá phức tạp	khá đơn giản

Bảng trên cho thấy trong bài này cây splay đều khá tốt ở ba tiêu chí thời gian, bộ nhớ và độ khó lập trình, nên rất phù hợp để dùng trong thi đấu.

4 Tổng kết

Từ phần phân tích trên, ta thấy cây splay có các ưu điểm sau.

1. Độ phức tạp thời gian thấp. Mọi thao tác cơ bản của cây splay đều có độ phức tạp khấu hao $O(\log n)$, đây là mức rất tốt trong các cấu trúc dữ liệu dạng cây.
2. Yêu cầu bộ nhớ không cao. Khác với cây đỏ đen phải lưu màu của mỗi nút, hay cây AVL phải lưu hệ số cân bằng, cây splay không cần lưu thông tin phụ để duy trì cân bằng.
3. Thuật toán đơn giản, dễ lập trình. Các thao tác cơ bản của cây splay đều dựa trên thao tác splay; trong thao tác này chỉ cần căn cứ vào vị trí của nút hiện tại để thực hiện các phép quay.

Mặc dù cây splay và cây AVL không chênh lệch nhiều về độ phức tạp thời gian, thậm chí trong một số tình huống cây splay có thể chậm hơn AVL, nhưng độ phức tạp lập trình của cây splay thấp hơn AVL rất nhiều. Trong thi đấu, sử dụng cây splay thường thuận lợi hơn khi viết và gỡ lỗi.

Trong thi tin học, không nên chỉ theo đuổi thuật toán có hiệu quả thời gian cao nhất. Cần tìm một điểm cân bằng giữa độ phức tạp thời gian, độ phức tạp bộ nhớ và độ phức tạp lập trình, rồi lựa chọn thuật toán hợp lý. Điều này đòi hỏi ta thường xuyên suy ngẫm và nghiên cứu sâu các thuật toán trong quá trình luyện tập, để khi thi có thể vận dụng thành thạo.

Tài liệu tham khảo

1. Fu Qingxiang, Wang Xiaodong. *Algorithms and Data Structures*. Publishing House of Electronics Industry, 1998.01.
2. Yan Weimin, Wu Weimin. *Data Structures*, second edition. Tsinghua University Press, 1992.06.
3. *Introduction to Algorithms*, Second Edition, 2001.

Phụ lục

1. Cài đặt thao tác splay và các thao tác cơ bản khác được tham chiếu trong tài liệu gốc qua tệp `Splay Tree.doc`.
2. Quá trình cụ thể của các thao tác cơ bản trên cây splay được minh họa trong tài liệu gốc qua hoạt hình `Splay Tree.htm`.
3. Với bài ví dụ trong bài viết, tác giả đã viết chương trình `Turnover.pas` sử dụng thuật toán cây splay.